SkillRAE: Support-Aware Skill Selection for Retrieval-Augmented Execution

Xiangcheng Meng
xiangchengmeng@link.cuhk.edu.cn
The Chinese University of Hong Kong, Shenzhen
Shenzhen, China

Shu Wang
shuwang3@link.cuhk.edu.cn
The Chinese University of Hong Kong, Shenzhen
Shenzhen, China

Yixiang Fang[†]
fangyixiang@cuhk.edu.cn
The Chinese University of Hong Kong, Shenzhen
Shenzhen, China

Abstract

For agents acting through Web tools and services, retrieving procedural resources entails selecting packages whose documented capabilities collectively support a task. We present SkillRAE, a retrieval-augmented execution framework that combines hierarchical candidate discovery with support-aware context compilation. The compiler links task requirements to source-grounded capabilities and jointly evaluates requirement coverage and community matching on the same fixed-size skill set. It preserves existing critical support when admitting additional coverage and supplies the selected packages with source-linked execution guidance. We develop Support-Interface Decomposition (SID), which compresses requirements supported by a single skill while retaining shared requirement identities. Under the stated support assumptions, this representation preserves coverage and matching feasibility and yields an exact algorithm parameterized by shared requirements and the matching quota. On SkillsBench, SkillRAE achieves the highest aggregate reward under both evaluated agent–model configurations, including a 10.3% relative improvement over the strongest external baseline with Codex/GPT-5.5. AgentSkillOS provides a second benchmark on which the complete framework remains competitive. The analysis identifies structure for joint selection, while the experiments evaluate the retrieval-to-execution framework. Code is available at https://github.com/honyeung1/SkillRAE-Official.

Keywords

agent skills, retrieval-augmented execution, evidence-guided skill selection

1 Introduction

Advances in large language models (LLMs) have enabled agents to take on an increasingly varied set of concrete tasks. Carrying out such tasks often requires interacting with tools or external environments, observing the effects of their actions, and adjusting subsequent steps. ReAct makes this reasoning–action loop explicit, while ToolPlanner studies planning and feedback in multi-round tool use [30, 33]. This kind of execution calls for procedural knowledge: when an operation applies, how to carry it out, and how to determine whether the requested outcome has been achieved. For agents that act through Web tools and services, this raises a retrieval problem: selecting procedural resources that jointly support a requested workflow within a bounded context. The agent consumes these resources as execution guidance, so topical relevance must be considered together with support for the operations the workflow requires. We study this problem through reusable skill packages.

Agent skills provide one way to externalize this procedural knowledge for reuse. In package-based systems, a skill can combine instructions, scripts, and reference material that an agent consults during execution, without updating the underlying model [14]. Reusable behavior also takes other forms: Voyager, for example, builds a library of executable code skills from experience [24]. Here, we focus on supplied skill packages as explicit resources that an agent can draw upon when facing a new task.

As skill libraries grow, exposing every package to an agent for every task becomes impractical [35]. Using skills at scale therefore involves two linked decisions: which packages to retrieve as candidates, and which of those candidates to make available during execution. In the bounded-skill setting we study, the first decision narrows the search space, while the second determines the procedural guidance the executor can actually consult. The resulting retrieval-and-selection process shapes which external procedures can influence the agent's actions.

Yet topical relevance alone does not establish that a skill documents what a task requires. A package can match the task's domain while leaving a necessary operation, an applicability constraint, or a completion check unspecified. For example, the SkillsBench GitHub analytics task selects pull requests by creation month but reports their current merge status [14]. A relevant guide need not document both conditions. This distinction also matters at the set level: individually relevant skills may repeat the same guidance, whereas another package may supply a missing capability (Figure 1). Replacing a selected package to obtain that capability can, in turn, remove support for a different requirement. The selection problem therefore concerns how documented capabilities collectively support task requirements, and what support is gained or lost when the selected set changes.

Recent work has begun to address these issues through richer representations of skills and their relationships. Graph-of-Skills retrieves dependency-aware bundles, while CaSKG uses textual counterfactual probes to calibrate procedural relations before retrieval [15, 16]. GoSkills organizes bounded skill groups around execution roles and visible task requirements [34], and SkillSmith compiles packages into operational interfaces for selective access at runtime [31]. These developments address dependencies, requirement coverage, and the organization of execution guidance. They motivate a more specific question: what evidence should justify changing a retrieved skill set, and how should that change account for the task requirements already supported?

[†]Corresponding author.

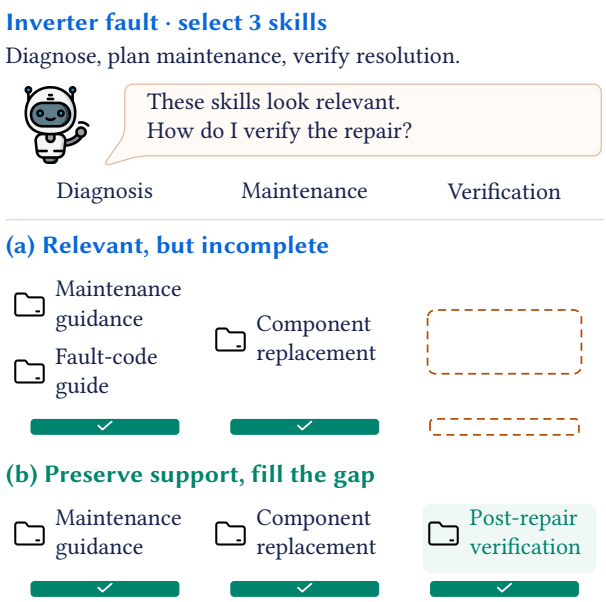


Figure 1: Relevant skills can leave task requirements unsupported. In this constructed example, three relevant skills provide overlapping diagnosis guidance but leave post-repair verification unsupported. Replacing the redundant guide adds verification support while preserving diagnosis and maintenance support under the same budget. Columns indicate supported requirements, not exclusive skill functions; checks denote documented support, not execution success. Icons: Smashicons and feen (Flaticon).

We study how to retrieve and select skill packages that collectively support a task under a fixed skill budget. This requires connecting two decisions: discovering useful candidates from the skill library and determining which combination of their documented capabilities supports the task's requirements. Retrieval provides both the candidates and an initial selection; requirement–capability evidence then informs whether that selection should be retained or revised. A revision should add support while preserving support for critical requirements already covered, whereas insufficient evidence should leave the initial selection intact.

We propose SkillRAE, a two-stage framework that combines hierarchical skill retrieval with support-aware context compilation. The first stage retrieves candidates and forms an initial skill set. The second stage links task requirements to capabilities documented in those candidates. It jointly evaluates critical coverage and the number of requirements supported by distinct skill communities, preserving existing critical support when adding new coverage. SID separates requirements supported by one candidate from those shared across candidates. The former can be summarized locally, while the latter retain their identities for joint selection. This decomposition preserves the joint optimum under its support assumptions. The complete selected packages and concise execution guidance are then passed to the agent. The same support relations inform both which packages enter the context and which procedures the executor is directed to consult.

On SkillsBench, SkillRAE achieves the highest aggregate reward under both evaluated agent–model configurations. With Codex CLI and GPT-5.5, it improves aggregate reward by 10.3% over Graph of Skills, the strongest external comparator. On AgentSkillOS, it ranks second with Codex and ties for the highest score with Grok. Across 87 SkillsBench tasks, SkillRAE supplies guidance for 28 tasks and changes the selected skill set for 11. The component comparisons evaluate the roles of hierarchical retrieval, context compilation, and capability granularity.

Our contributions are:

- **Execution-driven RAE framework.** We propose SkillRAE, a framework that integrates hierarchical candidate retrieval, support-aware skill selection, and source-linked guidance into task-specific execution contexts.
- **Joint selection and structural analysis.** We couple critical requirement coverage and community matching on the same fixed-budget skill set while preserving the initial set's critical support. SID separates private support from shared requirement identities and preserves the joint optimum under an explicit support model. The resulting exact algorithm has exponential dependence on shared requirements and the matching quota, rather than on private coverage or the total number of communities.
- **End-to-end empirical performance.** We evaluate SkillRAE on SkillsBench and AgentSkillOS under two agent–model configurations. The method achieves the highest SkillsBench reward in both configurations and competitive AgentSkillOS performance. Ablations and execution cases examine hierarchical retrieval, context compilation, and evidence granularity.

2 Related Work

Agent Skills. A skill is a reusable procedural artifact that externalizes task-focused know-how about when to act, how to execute, and how to assess task completion [14]. Existing work studies both how such skills are acquired and how they are used in subsequent tasks. Voyager accumulates executable code skills through interaction, while Trace2Skill consolidates execution experience into transferable skill packages [18, 24]. Such packages can combine instructions, scripts, and reference material, with SkillsBench evaluating their contribution to task performance [14]. Using these resources for a particular task involves finding suitable candidates, deciding which skills to use together, and organizing how they are used during execution. Content-based routing methods identify candidate skills, while dependency-aware retrieval methods assemble skills into bundles [16, 35]. AgentSkillOS further connects retrieval with multi-skill orchestration, while SkillSmith compiles skill packages into runtime interfaces for selective use [13, 31]. Together, these works show how skill discovery and the organization of procedural resources jointly shape what an agent can use for a task.

Tool and API Selection. Tool-use methods enable agents to select and invoke external operations and incorporate their outputs into task execution. These methods integrate tool calls into reasoning and generation through prompting and training [22, 33].

API retrieval methods extend this process to large tool libraries by identifying candidate APIs from their documentation and providing relevant interfaces for subsequent calls [19, 20]. Tool retrieval methods further improve matching between user requests and available tools through query reformulation and document enrichment [2]. Tool-use methods also connect selection with multi-step execution: hierarchical approaches combine API discovery with problem solving and self-reflection [3], while planning-based approaches use tool-use paths and feedback to improve task completion and instruction following [30]. These methods focus on selecting and coordinating callable operations, whereas SkillRAE retrieves procedural skill packages and organizes their documented capabilities into task-specific execution context.

Retrieval-Augmented Generation (RAG). RAG conditions language generation on evidence retrieved from external sources [12]. Graph-based and hierarchical retrieval methods exploit entity relations and summaries at multiple levels to find relevant evidence [4, 21]. ArchRAG builds hierarchical indexes over attributed communities, while BookRAG combines document hierarchies with entity relations to guide retrieval [25, 26]. A unified evaluation framework compares graph construction, indexing, and retrieval operators and examines their effects on downstream question answering [36]. Adaptive retrieval methods connect evidence acquisition with ongoing generation and reasoning, allowing intermediate outputs to guide subsequent retrieval [11, 23]. Retrieved material can also be selected and reorganized before it enters the model's context. RECOMP and LongLLMLingua reduce context through compression, while TRACE assembles supporting information into reasoning chains [5, 10, 32]. These approaches connect evidence discovery with context construction. SkillRAE studies this connection for procedural skill packages, focusing on how their documented capabilities support specific execution requirements.

Retrieval-Augmented Execution (RAE). RAE extends retrieval augmentation from knowledge-grounded generation to agent execution. It retrieves and organizes procedural resources, such as skills, tools, and APIs, to support task completion. Skill-routing methods identify candidates through skill contents and distinctions between neighboring capabilities [28, 35]. Structural retrieval methods use dependencies and calibrated skill relations to assemble execution bundles [15, 16]. Set-selection methods address query-conditioned compatibility and redundancy [27, 29]. BPS balances capability benefits against context penalties under a token budget, using a model learned from execution outcomes [1]. Beyond set selection, related work examines how skills are organized and used during execution. GoSkills organizes skills into groups, tracks uncovered task requirements, and constructs bounded context with explicit execution roles [34]. SkillSmith compiles skill packages into interfaces for selective runtime use [31]. SkillApt uses paired execution outcomes with and without a skill to decide whether to activate it [6]. Related theoretical formulations include coverage optimization under matchoid constraints [8] and rainbow matching [7]. SkillRAE couples these two types of quantities through the selected skills: each selected package contributes coverage and activates requirement–community support. SID compresses requirements with a unique supplying skill while retaining shared identities, preserving coverage and matching feasibility under one skill budget. The resulting exact algorithm depends exponentially on the shared requirements and matching quota, even when private coverage and the number of communities grow. This structural analysis complements the hierarchical retrieval and source-linked guidance of the complete RAE framework.

3 Method: SkillRAE

3.1 Overview

Given a task request q and a repository of reusable skills, a RAE system retrieves relevant skills and compiles a task-specific context for an executor to complete q. Following this framework, SkillRAE constructs the context from a fixed-size set of complete skill packages and optional task-specific guidance. As illustrated in Figure 2, the offline stage prepares a hierarchical retrieval graph and source-grounded capability representations from the skill library. The online stage performs skill retrieval and context compilation to determine the final skill set and prepare its execution guidance.

Specifically, the offline stage constructs two complementary representations of the skill library. A hierarchical retrieval graph G_R organizes communities, skills, and source blocks to support candidate discovery at multiple levels. Separately, the capability representation G_E records the operations documented in each skill and retains the source passages used to assess whether those operations support a task's requirements. Both source blocks and capability records remain linked to their originating skills, preserving complete skill packages as the units supplied to the executor. These parent links connect accepted task support to the skill memberships in G_R, allowing the compiler to evaluate coverage and community matching on the same selected skills.

In the online stage, SkillRAE combines retrieval signals from the levels of G_R to form a candidate pool and an initial skill set S_0. The context compiler then uses G_E to assess how the candidates' capabilities support the task requirements. It selects skills jointly, prioritizing critical requirement coverage and then the number of requirements that can be matched to distinct supporting communities. Any replacement must preserve support for the critical requirements already covered by S_0 and add support for further critical requirements. If no set qualifies, S_0 is retained. Support-Interface Decomposition (SID) identifies a value-preserving decomposition of this joint selection problem under a stated support model. The compiler uses support relations within the final set to prepare source-linked guidance when available. It delivers the complete selected packages and any resulting guidance as the task-specific context $C(q)$.

3.2 Offline Stage

Hierarchical retrieval graph G_R. In the offline stage, we construct a hierarchical retrieval graph over communities, skills, and source blocks to support candidate discovery at different levels of granularity. Formally, we write

$$G_R = (V_G, V_S, V_B, E_G, E_B), \tag{1}$$

where V_G, V_S, and V_B denote the sets of communities (L2), skills (L1), and source blocks (L0), respectively. The edges $E_G \subseteq V_G \times V_S$ encode community membership, while $E_B \subseteq V_S \times V_B$ link source blocks to their originating skills. Each skill is assigned to one community, and

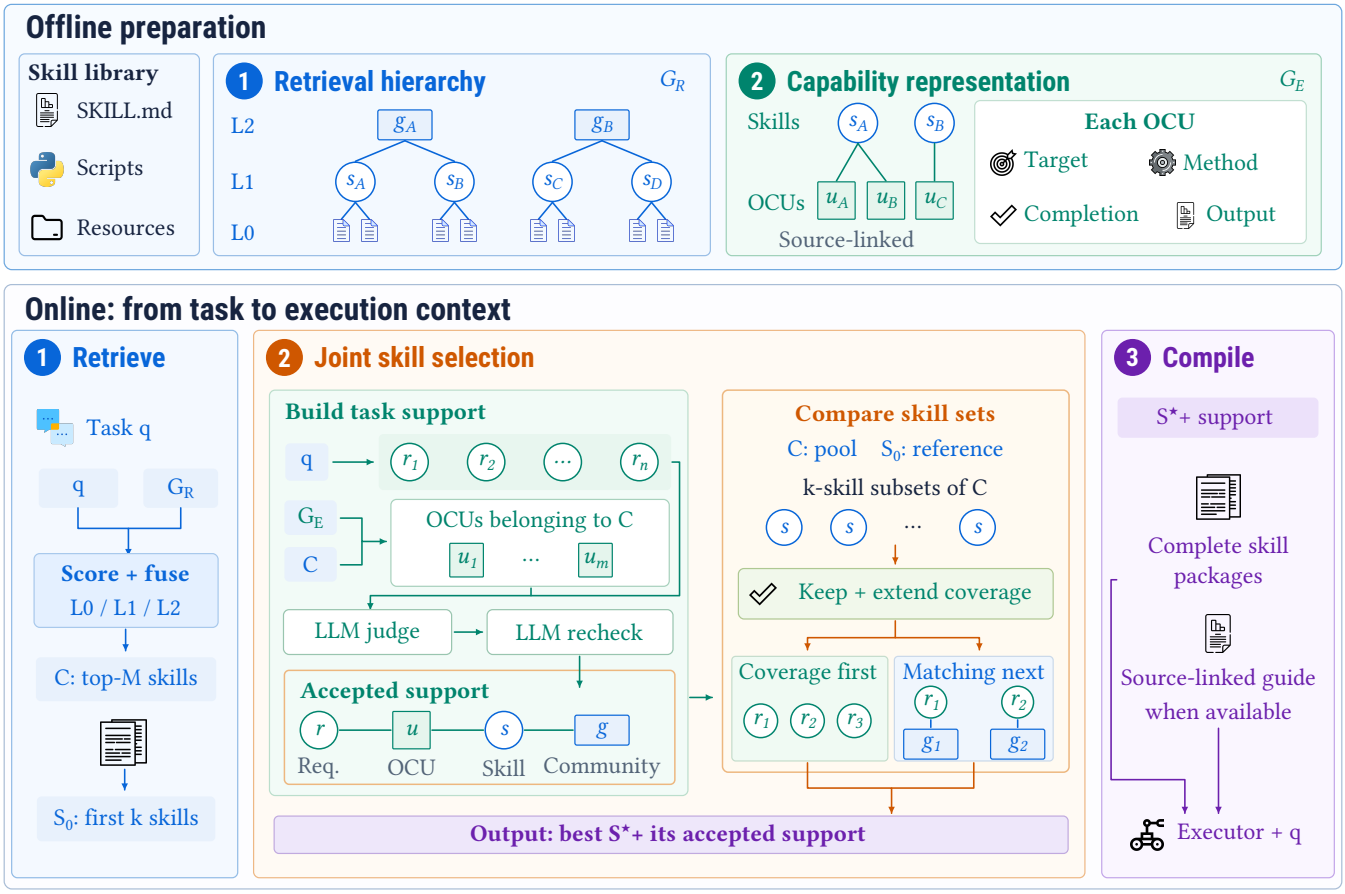


Figure 2: SkillRAE overview. Offline construction prepares the retrieval hierarchy and source-linked OCUs. Online retrieval forms candidates C and a reference set S_0. Reviewed requirement–OCU support guides fixed-budget selection that preserves and extends critical coverage, prioritizing coverage then community matching; otherwise S_0 is retained. Complete packages and optional guidance reach the executor. Figure 3 illustrates the structure underlying joint selection.

each source block has exactly one parent skill. This organization supports retrieval over related communities, individual skills, and local content while preserving the provenance of each source block.

During construction, we use skill descriptions to represent L1 skill nodes and extract L0 source blocks from their Markdown documentation. Block extraction follows document structure, preserving procedural paragraphs, complete lists, and code blocks together with adjacent explanatory text. We encode both skill descriptions and source blocks using the same embedding model, including skill names and section headings in the block inputs. Each block retains its original text, source location, and parent-skill association, even when multiple windows are used for encoding.

To form communities, we cluster the L1 skill embeddings using k-means, assigning each skill to one community. Each community is represented by the normalized centroid of its member embeddings. These community representations support task matching over related skills and provide an additional retrieval signal for their members. The compiler also retains each skill's community membership when it constructs requirement–community relations from accepted OCU support.

Capability representation G_E. Alongside G_R, we construct G_E to represent the operations documented in each skill as operational capability units (OCUs). Each OCU describes an operation through four attributes: Target, Method, Completion, and Output. Target names the documented operation; Method describes how it is performed. Completion states its declared completion criterion, and Output identifies the expected artifact or effect. It links these descriptions to supporting source passages and retains its association with the parent skill. Online matching uses these attributes together to assess an operation against a task requirement, including its material conditions. This gives the compiler a procedure-level basis for support judgments while retaining the source context needed to check them.

We construct OCU candidates around source passages describing an operation, adding related procedural and completion evidence

from the same skill. An LLM assesses whether each candidate describes a complete operation and whether the supplied evidence supports all four components. A separate LLM review then checks the proposed unit against the original sources for missing steps or unsupported components. We retain only units accepted by both assessments for online requirement–capability matching. For example, a reflow-maintenance OCU links the computation of time above liquidus to its documented procedure, completion criterion, and duration output.

3.3 Online Stage

In the online stage, SkillRAE retrieves candidate skills from G_R for a task request q and forms an initial skill set S_0. The context compiler uses G_E to establish task support, jointly select skills under the fixed budget, and prepare task-specific guidance when available. The executor receives the complete selected skill packages and any resulting guidance to carry out q.

3.3.1 Skill Retrieval.

Top-down retrieval. Our skill retrieval method combines top-down signals from communities with bottom-up signals from source blocks. We encode the task request q using the same embedding model as in the offline stage. We score each L2 community by the cosine similarity between the task embedding and the community's centroid. The member skills of the highest-scoring communities form $\mathcal{B}(q)$ and receive a community-level boost in the subsequent score fusion.

Bottom-up retrieval. We match q to L0 source blocks and project their scores to their parent skills. Let $\sigma(q, b)$ denote a block's task similarity, computed as the maximum cosine similarity over its encoding windows. For the N highest-scoring blocks $\mathcal{T}_N(q)$, the aggregated score of skill s is

$$\widetilde{\ell_0}(s, q) = \sum_{b \in \mathcal{T}_N(q)} \mathbf{1}\{(s, b) \in E_B\}\, \sigma(q, b). \tag{2}$$

We obtain $\ell_0(s, q)$ by min–max normalizing these sums over skills with retained blocks, assigning zero to the remaining skills. The direct skill-level signals $\ell_1(s, q)$ and $p(s, q)$ are the min–max normalized cosine similarities of the task with the skill description and name, respectively.

Candidate formation. We combine skill-level similarity, bottom-up block scores, and the skill-name score with the top-down community weighting:

$$r(s, q) = \big(\alpha\ell_1(s, q) + \beta\ell_0(s, q) + \gamma p(s, q)\big)\big(1 + \lambda \mathbf{1}\{s \in \mathcal{B}(q)\}\big), \tag{3}$$

where α, β, and γ are fixed weights, and λ controls the community-level boost. The M highest-ranked skills form the candidate pool C, and its first k skills form the initial set S_0:

$$C = \mathrm{TopM}_{s \in V_S}\, r(s, q), \qquad S_0 = \mathrm{Topk}_{s \in C}\, r(s, q). \tag{4}$$

The retriever passes C to the context compiler together with S_0, which provides the protected-support reference and fallback selection. The compiler uses the candidates' capability records in G_E to assess task support and determine the final skill set.

3.3.2 Context Compilation.

Given the candidate pool C and initial set S_0, we aim to construct a task-specific context comprising complete skill packages and optional execution guidance. We first establish support relations between task requirements and candidate OCUs in G_E. The compiler uses these relations to optimize requirement coverage and community matching on the same size-k skill set, subject to preserving the critical support of S_0. We then analyze the structure of this joint decision through SID. The compiler uses the support relations within the final set to prepare source-linked guidance when available.

Requirement–capability binding. We extract task requirements from q, retaining the original passages that describe actions, outputs, and constraints. We mark critical requirements using lexical cues for explicit constraints and core task actions. For each requirement, we rank OCUs from the candidate skills in C by TF–IDF similarity between the requirement text and the OCU descriptions and source passages. We retain a bounded shortlist for each requirement, limiting the number of OCUs contributed by any one skill. The resulting requirement–OCU pairs are passed to support assessment.

For each nominated pair, an LLM assesses whether the OCU's documented procedure can satisfy the cited requirement. The assessment checks the required action, applicable input conditions, expected output, and any required completion or validation check. A separate LLM review reexamines positive judgments against the task requirement and the same source evidence. We accept a binding only when both assessments support the requirement without unresolved or contradicted material conditions. Each accepted binding retains the requirement, OCU, parent skill, and supporting source references. The parent link associates each accepted binding with the skill's community membership in G_R. These identities allow the selector to compare different skill sets using the same support records.

Joint skill selection. Selecting a skill consumes one budget slot and activates its accepted requirement support. Coverage and community matching are therefore evaluated on the same selected parents. Let $P(S)$ contain the accepted support records available within a skill set S, with all source skills included in S. For each record p, let $Q(p)$ denote its supported requirements and $v(p)$ indicate recorded completion or validation evidence. We define

$$\begin{aligned} R(S) &= \bigcup_{p \in P(S)} Q(p), \\ H(S) &= R(S) \cap R_{\mathrm{crit}}, \\ V(S) &= \bigcup_{\substack{p \in P(S) \\ v(p)=1}} Q(p), \end{aligned} \tag{5}$$

where R_{crit} is the set of critical task requirements. To distinguish support drawn from different skill communities, we connect each supported requirement to the community labels carried by its support records. Maximum bipartite matching gives $m_H(S)$ for critical requirements and $m_R(S)$ for all supported requirements. These quantities count requirements that can be assigned to distinct supporting communities. Each matched community must support its assigned requirement through a selected skill. Among sets with

equal critical coverage, this favors support distributed across communities without counting an unsupported community as a benefit.

An admissible selection must contain k skills, preserve the critical requirements covered by S_0, and support at least one additional critical requirement. It must also admit a positive critical requirement–community matching. The admissible sets are therefore

$$\mathcal{F} = \left\{ S \subseteq C \;\middle|\; \begin{array}{l} |S| = k, \quad H(S_0) \subseteq H(S), \\ H(S) \setminus H(S_0) \neq \emptyset, \quad m_H(S) > 0 \end{array} \right\}. \tag{6}$$

Within $\mathcal{F}$, we first maximize critical coverage and then critical matching. Even in a restricted support class where coverage and matching can each be optimized in polynomial time, deciding whether one size-k skill set attains both targets is NP-complete (Appendix A.2).

Support-Interface Decomposition. SID analyzes selection within the retrieved candidate pool C, after requirement–OCU judgments have been accepted. Here, community memberships identify which communities supply accepted task support. We treat these judgments as fixed. Let $\mathcal{R}_s$ contain the critical requirements supported by skill s, and let $\mathcal{G}_s$ contain its community memberships. Under unary activation and parent-consistent memberships, selecting s activates

$$E_s = \mathcal{R}_s \times \mathcal{G}_s. \tag{7}$$

The analysis assumes unit skill costs, unweighted coverage, and monotone support without additional prerequisites or conflicts. It allows arbitrary community memberships, including the prototype's single-community assignment.

Some requirements have only one supplying skill in C; others are supported by several candidates. Let T contain the shared critical requirements, and write

$$A_s = \mathcal{R}_s \cap T, \qquad P_s = \mathcal{R}_s \setminus T, \qquad p_s = |P_s|. \tag{8}$$

The private sets P_s are disjoint, so selecting s contributes p_s private requirements without overlap with another parent. Shared requirements are assigned to selected supplying skills, with each requirement credited once. The assignment must also support the community-matching witness: fixing ownership before matching can discard a feasible solution. Private requirements contribute local coverage and matching capacity. Jointly choosing ownership and matching preserves the value and feasibility of each selected skill set. Figure 3 illustrates this correspondence on a complete three-skill instance.

Theorem (Support-Interface Decomposition). Under the preceding model, let $t = |T|$, let h be a critical matching quota, and let N be the explicit input size. An optimal solution to

$$\begin{array}{l} \max\limits_{S \subseteq C} |H(S)| \\ \text{subject to } |S| = k, \quad H(S_0) \subseteq H(S), \quad m_H(S) \geq h \end{array} \tag{9}$$

can be found deterministically in $2^{O(t+h)}\,\mathrm{poly}(N)$ time, or the instance can be declared infeasible. Private requirement counts and the total number of communities need not enter the exponent. Thus, additional private requirements enlarge local values rather than the shared state space; combinatorial coordination is concentrated on shared identities and the matching quota. Appendix A.1 gives the value-preserving construction and proof. Varying the quota recovers the coverage-first, matching-second optimum with strict-gain admission and baseline fallback. The guarantee concerns these two leading objectives; later tie-breaks remain part of the prototype's complete ordering.

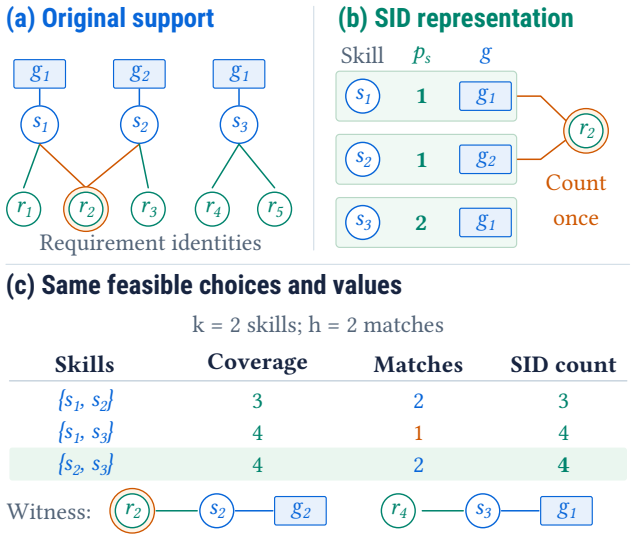


Figure 3: SID on one illustrative instance ($k = 2$, $h = 2$, no protected requirements). Private requirements become counts p_s; shared r_2 retains its identity. The table lists every size-two set: coverage counts and maximum matching sizes agree across representations. The highlighted optimum covers four requirements; the other four-coverage set fails the matching quota. The witness for the optimum credits r_2 to s_2 and uses a private requirement of s_3.

Prototype selection. The prototype solves the joint decision by enumerating the $\binom{M}{k}$ skill sets within C. It compares admissible sets using the complete lexicographic objective

$$\begin{aligned} J(S) = \big(&|H(S)|, m_H(S), m_R(S), |R(S)|, |V(S)|, \\ &-|S \setminus S_0|, -\sum_{s \in S} \rho(s), -\boldsymbol{\rho}(S)\big), \end{aligned} \tag{10}$$

where $\rho(s)$ is the first-stage retrieval rank and $\boldsymbol{\rho}(S)$ lists these ranks in ascending order. The objective first prioritizes critical coverage and requirement–community matching. It then favors total and validation-supported coverage, fewer replacements, and higher retrieval ranks. The final set is

$$S^\star = \begin{cases} \arg\max^{\mathrm{lex}}_{S \in \mathcal{F}} J(S), & \mathcal{F} \neq \emptyset, \\ S_0, & \mathcal{F} = \emptyset. \end{cases} \tag{11}$$

A fixed candidate order resolves any remaining ties. This search is exact within C and permits multiple skill replacements. This enumeration is an uncompressed exact realization of the decision analyzed by SID. The parameterized running-time bound applies to the structured solver in Appendix A.1, rather than to the deployed enumeration.

Context organization. We compile a bounded guide from accepted bindings within $S^\star$. For the initial set, an LLM selects non-redundant bindings that provide task-specific procedures and completion checks, and a separate review validates the selection. When the set changes, bindings that support newly covered critical requirements through incoming skills take priority. Still-applicable bindings from the initial guide can fill the remaining guide slots. Each guide entry links a task requirement to operational passages and completion evidence in a selected skill package. Writing the resulting guide as $D(q, S^\star)$, we deliver

$$C(q) = \big(q, \{\text{Package}(s) : s \in S^\star\}, D(q, S^\star)\big) \tag{12}$$

to the executor, where $\text{Package}(s)$ is the complete skill package. The guide is bounded by a token budget and may be empty. The same context organization applies when $S^\star = S_0$.

Shared-Support Optimality Certification. Shared-Support Optimality Certification (SSOC) analyzes the same leading selection objective when some support judgments are unresolved. It determines whether an incumbent remains eligible and optimal under every consistent completion of the shared support records. The current pipeline resolves its nominated judgments before selection; it does not use SSOC for adaptive querying or early stopping.

4 Experiments

We evaluate three questions: whether SkillRAE improves end-to-end task performance over existing skill-selection methods; how hierarchical retrieval and evidence-grounded compilation contribute to that performance; and how admitted capability evidence changes the inputs available to the executor. Tables 1–3 address these questions in turn.

4.1 Experimental setup

Benchmarks and metrics. We use two execution benchmarks with their respective public skill libraries. SkillsBench [14] comprises 87 evaluated tasks and a 207-skill retrieval pool. Its `gh-repo-analytics` task requests a GitHub repository report with explicit time filters, bug-label rules, and merge-duration units. Its aggregate reward is the sum of official task rewards, including partial credit, with a maximum of 87. AgentSkillOS [13] comprises 30 tasks and a complete 200-skill pool. We report the sum of its official weighted objective scores, with a maximum of 3000. Table 1 reports raw scores and benchmark-normalized percentages. Macro is the equal-weight mean of the two normalized percentages.

Agent–model configurations. We use Codex CLI with GPT-5.5 and Grok Build CLI with Grok-4.6.

SkillRAE settings. We retrieve 20 candidate skills and select five complete packages for execution. Source-linked guidance is limited to 256 tokens and two entries.

Comparators. We compare with Vanilla Top-k, SkillRouter [35], Graph of Skills (GoS) [16], CaSKG [15], and a benchmark-native reference. Native exposes benchmark-provided public skill bundles on SkillsBench and uses the skill-tree/DAG workflow on AgentSkillOS. We evaluate these systems with their respective retrieval and context-construction interfaces.

Reporting protocol. Table 1 summarizes official task-level scores for each method. Reporting details are provided in Appendix A.5. On AgentSkillOS, we report the aggregate official score over 30 tasks for each method. For Grok SkillsBench, Table 1 aggregates the available official task rewards.

4.2 End-to-end performance

SkillsBench. SkillRAE has the highest aggregate reward under both agent–model configurations. With Codex/GPT-5.5, it scores 47.0245 versus 42.6208 for GoS, the strongest external comparator: a gain of 4.4037 reward units, or 10.3% relative improvement. With Grok, it scores 51.2300 versus 44.8478 for SkillRouter, a margin of 6.3822 reward units. The corresponding improvements are 5.06 and 7.34 percentage points of the benchmark maximum. The advantage is consistent across both evaluated agent–model configurations.

AgentSkillOS and cross-benchmark performance. With Codex, SkillRAE scores 2904.71 on AgentSkillOS, second to GoS at 2936.38. With Grok, SkillRAE and SkillRouter tie at 2963.45, ahead of GoS at 2933.45. SkillRAE achieves the highest Macro score under both configurations, 75.44 and 78.83, combining its SkillsBench advantage with competitive performance on AgentSkillOS. We examine the different activation regimes of the two benchmarks below.

4.3 Contribution of the core components

We evaluate three ablations on SkillsBench with Codex/GPT-5.5 (Table 2): hierarchical candidate retrieval, evidence-grounded compilation, and the granularity of capability evidence.

Hierarchical candidate retrieval. The *w/o hierarchical retrieval* control uses flat L_1-only candidate retrieval while retaining the full second stage. Its reward is 33.7130, a decrease of 13.3115 from the full method. This contrast supports hierarchical candidate retrieval as an upstream component of evidence-grounded compilation.

Evidence-grounded compilation. The *w/o Stage II* control directly uses the first-stage selection, removing requirement–OCU binding, support-aware set selection, and compiled guidance together. Reward falls to 31.3115, a decrease of 15.7130, the largest observed drop among these controls. This contrast supports evidence-grounded compilation as an integrated component of SkillRAE.

OCU-level evidence. The *w/o OCU-level evidence* control uses whole-skill text for support judgments while retaining the nominated requirement–parent-skill pairs and the binding and set-selection machinery. It scores 36.8661, 10.1584 below the full method. The contrast favors OCU-level evidence for assessing requirement support in Stage II.

Together, these ablations support the combination of hierarchical candidate discovery and OCU-grounded context compilation.

4.4 Capability evidence and executor behavior

From capability support to execution inputs. Table 3 summarizes how capability support changes execution inputs on SkillsBench with Codex/GPT-5.5. Admitted requirement–OCU support is available for 58 of 87 tasks. Of these, 11 receive both a changed skill set and guidance, and 17 receive guidance with the original skill set. In total, 28 tasks receive guidance. These outcomes distinguish

Table 1: End-to-end performance on SkillsBench and AgentSkillOS. Macro is the mean of the two benchmark-normalized percentages. Bold and underline denote the best and second-best results within each agent–model configuration, respectively.

Backbone + LLM	Method	SkillsBench / 87	AgentSkillOS / 3000	Macro (%)
Codex CLI + GPT-5.5	Vanilla Top-k	40.1988 (46.21%)	2861.10 (95.37%)	70.79
	SkillRouter	40.8330 (46.93%)	2802.34 (93.41%)	70.17
	Native / benchmark method	41.1778 (47.33%)	2903.05 (96.77%)	72.05
	Graph of Skills	<u>42.6208 (48.99%)</u>	**2936.38 (97.88%)**	<u>73.43</u>
	CaSKG	41.2800 (47.45%)	2892.02 (96.40%)	71.92
	SkillRAE	**47.0245 (54.05%)**	<u>2904.71 (96.82%)</u>	**75.44**
Grok Build CLI + Grok-4.6	Vanilla Top-k	42.0078 (48.28%)	2773.66 (92.46%)	70.37
	SkillRouter	<u>44.8478 (51.55%)</u>	**2963.45 (98.78%)**	<u>75.17</u>
	Native / benchmark method	43.7978 (50.34%)	2651.22 (88.37%)	69.36
	Graph of Skills	44.6308 (51.30%)	<u>2933.45 (97.78%)</u>	74.54
	CaSKG	43.1200 (49.56%)	2915.80 (97.19%)	73.38
	SkillRAE	**51.2300 (58.89%)**	**2963.45 (98.78%)**	**78.83**

Table 2: SkillsBench ablations with Codex CLI and GPT-5.5. Each control modifies hierarchical retrieval, Stage II compilation, or evidence granularity, as defined in the text.

Variant	Reward / 87
Full SkillRAE	47.0245
w/o hierarchical retrieval	33.7130
w/o Stage II	31.3115
w/o OCU-level evidence	36.8661

Table 3: Evidence support and executor-input changes on SkillsBench (Codex CLI, GPT-5.5).

Metric	Tasks
Tasks with admitted OCU support	58/87
Tasks with skill-set changes	11/87
Tasks with guidance only	17/87

capability support from its two uses in context compilation: skill-set selection and execution guidance.

Guidance and skill access. Nonempty guides contain 103–249 tokens (median 144). A guide-associated skill in the selected set is opened on 25 of 87 tasks. These observations document concise guidance and subsequent access to relevant skill packages.

Case study: capability evidence in execution. We examine two execution cases to illustrate how capability evidence relates to skill use and task outcomes. In `manufacturing-equipment-maintenance`, the full method and the whole-skill evidence control mount the same five skills. The full guide links the public time-above-liquidus requirement to a maintenance OCU. The executor reads the maintenance skill, produces the five requested JSON outputs, and receives reward 1. The control reads the compliance toolkit, has no recorded maintenance-skill read, and receives reward 0 with required outputs missing. This case traces the connection between capability evidence, skill access, and delivered artifacts. A contrasting `grid-dispatch-operator` case records an additional skill read with the full method but a lower reward (0 versus 1). Together, the cases illustrate execution behavior beyond the identity of the selected skills.

Activation across benchmarks. For Codex on AgentSkillOS, one of 571 nominated bindings is admitted, one task receives guidance, and the retrieved skill sets are retained throughout. The Grok evaluation also produces one nonempty capability guide, with task-level objective scores matching SkillRouter. Stage II therefore changes execution inputs less frequently on AgentSkillOS than on SkillsBench. SkillsBench provides the component comparisons and observations of both set selection and guidance, while AgentSkillOS evaluates the complete framework in a regime with little intervention.

5 Limitations

Our evaluation covers two benchmarks, with SkillsBench also used for method development and selection. These results do not establish broad generalization or isolate the benefits of individual components. LLM-based support judgments may be incorrect or incomplete, and preserving documented requirement coverage does not guarantee successful execution. SID provides an exact joint-selection guarantee within a fixed candidate pool under the stated support assumptions, including unary activation, parent-consistent memberships, and unit skill costs. Its parameterized complexity bound applies to the structured solver, while the evaluated prototype uses enumeration. The analysis does not establish retrieval optimality or end-to-end performance guarantees.

6 Conclusion

SkillRAE connects hierarchical skill retrieval with support-aware context compilation. It uses source-grounded capability relations to select complete skill packages and prepare execution guidance. Its selection mechanism evaluates critical coverage and community matching on the same skill set while preserving existing critical support. SID identifies a value-preserving decomposition of this decision: private support can be summarized locally, whereas shared requirements retain their identities for joint coordination. The resulting exact guarantee applies within the retrieved candidate pool

under the stated support assumptions. Experiments on two benchmarks show strong end-to-end performance and document how capability evidence enters skill selection and guidance. For agents using Web tools and services, this connects procedural-resource retrieval to the joint selection of task-supporting execution context.

References

[1] Yu Chen, Ruishuo Chen, Xun Wang, Zhuoran Li, and Longbo Huang. 2026. Optimal Skill Selection for LLM Agents with Provable Bicriteria Guarantees. *arXiv preprint arXiv:2608.19993* (2026). doi:10.48550/arXiv.2608.19993

[2] Yanfei Chen, Jinsung Yoon, Devendra Singh Sachan, Qingze Wang, Vincent Cohen-Addad, Mohammadhossein Bateni, Chen-Yu Lee, and Tomas Pfister. 2024. Re-Invoke: Tool Invocation Rewriting for Zero-Shot Tool Retrieval. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. Association for Computational Linguistics, Miami, Florida, USA, 4705–4726. doi:10.18653/v1/2024.findings-emnlp.270

[3] Yu Du, Fangyun Wei, and Hongyang Zhang. 2024. AnyTool: Self-Reflective, Hierarchical Agents for Large-Scale API Calls. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 11812–11829. https://proceedings.mlr.press/v235/du24h.html

[4] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. 2024. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *arXiv preprint arXiv:2404.16130* (2024). arXiv:2404.16130 [cs.CL] doi:10.48550/arXiv.2404.16130

[5] Jinyuan Fang, Zaiqiao Meng, and Craig Macdonald. 2024. TRACE the evidence: Constructing knowledge-grounded reasoning chains for retrieval-augmented generation. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 8472–8494. doi:10.18653/v1/2024.findings-emnlp.496

[6] Shuang Guo. 2026. SkillApt: Learning When to Activate Agent Skills from Counterfactual Evidence. *arXiv preprint arXiv:2609.26863* (2026). doi:10.48550/arXiv.2609.26863

[7] Sushmita Gupta, Sanjukta Roy, Saket Saurabh, and Meirav Zehavi. 2017. Parameterized Algorithms and Kernels for Rainbow Matching. In *42nd International Symposium on Mathematical Foundations of Computer Science (Leibniz International Proceedings in Informatics, Vol. 83)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 71:1–71:13. doi:10.4230/LIPIcs.MFCS.2017.71

[8] Chien-Chung Huang and Justin Ward. 2022. FPT-Algorithms for the ℓ-Matchoid Problem with a Coverage Objective. doi:10.48550/arXiv.2011.06268 arXiv:2011.06268v3.

[9] Russell Impagliazzo, Ramamohan Paturi, and Francis Zane. 2001. Which Problems Have Strongly Exponential Complexity? *J. Comput. System Sci.* 63, 4 (2001), 512–530. doi:10.1006/jcss.2001.1774

[10] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024. LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 1658–1677. doi:10.18653/v1/2024.acl-long.91

[11] Zhengbao Jiang, Frank F Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Active retrieval augmented generation. In *Proceedings of the 2023 conference on empirical methods in natural language processing*. 7969–7992. doi:10.18653/v1/2023.emnlp-main.495

[12] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[13] Hao Li, Chunjiang Mu, Jianhao Chen, Siyue Ren, Zhiyao Cui, Yiqun Zhang, Lei Bai, and Shuyue Hu. 2026. Organizing, Orchestrating, and Benchmarking Agent Skills at Ecosystem Scale. *arXiv preprint arXiv:2603.02176* (2026). doi:10.48550/arXiv.2603.02176

[14] Xiangyi Li, Yimin Liu, Wenbo Chen, Bingran You, Zonglin Di, Yifeng He, Shenghan Zheng, Kyoung Whan Choe, Jiankai Sun, Shuyi Wang, Chujun Tao, Binxu Li, Xuandong Zhao, Hejia Geng, Xiaojun Wu, Junwei Zhou, Xiaokun Chen, Hanwen Xing, Yubo Li, Qunhong Zeng, Di Wang, Yuanli Wang, Roey Ben Chaim, Penghao Jiang, Haotian Shen, Luyang Kong, Xinyi Liu, Runhui Wang, Xuanqing Liu, Jiachen Li, Xin Lan, Yueqian Lin, Wengao Ye, Junwei He, Songlin Li, Yue Zhang, Yipeng Gao, Yijiang Li, Ze Ma, Liqiang Jing, Tianyu Wang, Kaixin Li, Yiqi Xue, Haoran Lyu, Yizhuo He, Yuchen Tian, Shutong Wu, Bowei Wang, Yixuan Gao, Bo Chen, Litong Liu, Sikai Cheng, Jiajun Bao, Shuaicheng Tong, Shuwen Xu, Terry Yue Zhuo, Tinghan Ye, Qi Qi, Miao Li, Longtai Liao, Zelin Tan, Chang Shi, Xilin Tang, Srinath Tankasala, Boqin Yuan, Yaoyao Qian, Jianhong Tu, Chenguang Wang, Yizhou Sun, Wei Wang, Aaron Taylor, Ziyue Yang, Changkun Guan, Zhikang Dong, Xinyu Zhang, Steven Dillmann, Han-chung Lee, and Dawn Song. 2026. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks. *arXiv preprint arXiv:2602.12670* (2026). doi:10.48550/arXiv.2602.12670

[15] Zhiyuan Li, Linyuan Gao, Xuechun Ding, Hongwei Chen, Yuan Wu, and Yi Chang. 2026. CaSKG: Counterfactual-Causal Skill Graphs for Scalable Agent Skill Retrieval. *arXiv preprint arXiv:2608.25500* (2026). doi:10.48550/arXiv.2608.25500

[16] Dawei Liu, Zongxia Li, Hongyang Du, Xiyang Wu, Shihang Gui, Yongbei Kuang, and Lichao Sun. 2026. Graph-of-Skills: Dependency-Aware Structural Retrieval for Massive Agent Skills. *arXiv preprint arXiv:2604.05333* (2026). doi:10.48550/arXiv.2604.05333

[17] Moni Naor, Leonard J. Schulman, and Aravind Srinivasan. 1995. Splitters and Near-Optimal Derandomization. In *Proceedings of the 36th Annual Symposium on Foundations of Computer Science*. IEEE, 182–191. doi:10.1109/SFCS.1995.492475

[18] Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Erchao Zhao, Xiaoxi Jiang, and Guanjun Jiang. 2026. Trace2Skill: Distill Trajectory-Local Lessons into Transferable Agent Skills. *arXiv preprint arXiv:2603.25158* (2026). doi:10.48550/arXiv.2603.25158

[19] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024. Gorilla: Large Language Model Connected with Massive APIs. In *Advances in Neural Information Processing Systems*. https://proceedings.neurips.cc/paper_files/paper/2024/hash/e4c61f578ff07830f5c37378dd3ecb0d-Abstract-Conference.html

[20] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2024/hash/28e50ee5b72e90b50e7196fde8ea260e-Abstract-Conference.html

[21] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. 2024. RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. In *International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2024/hash/8a2acd174940dbca361a6398a4f9df91-Abstract-Conference.html

[22] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*. https://proceedings.neurips.cc/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html

[23] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2023. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*. 10014–10037. doi:10.18653/v1/2023.acl-long.557

[24] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv preprint arXiv:2305.16291* (2023). arXiv:2305.16291 [cs.AI] doi:10.48550/arXiv.2305.16291

[25] Shu Wang, Yixiang Fang, Yingli Zhou, Xilin Liu, and Yuchi Ma. 2026. ArchRAG: Attributed Community-based Hierarchical Retrieval-Augmented Generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 15868–15876. doi:10.1609/aaai.v40i19.38619

[26] Shu Wang, Yingli Zhou, and Yixiang Fang. 2025. BookRAG: A Hierarchical Structure-aware Index-based Approach for Retrieval-Augmented Generation on Complex Documents. *arXiv preprint arXiv:2512.03413* (2025). arXiv:2512.03413 [cs.CL] doi:10.48550/arXiv.2512.03413

[27] Zifei Wang, Wei Wen, Qiang Ji, Keyu Chen, Ruizhi Qiao, and Xing Sun. 2026. Skill Is Not Document: Query-Conditioned Compatibility for LLM Agent Skill Routing. *arXiv preprint arXiv:2606.03565* (2026). doi:10.48550/arXiv.2606.03565

[28] Zifei Wang, Wei Wen, Qiang Ji, and Ruizhi Qiao. 2026. Skills Know Their Neighbors: Cluster-Contrastive Capability Pages for Skill Retrieval. *arXiv preprint arXiv:2608.04482* (2026). doi:10.48550/arXiv.2608.04482

[29] Wang Wei, Tiankai Yang, Samyadeep Basu, Hongjie Chen, Yue Zhao, Zhengzhong Tu, Xiyang Hu, Franck Dernoncourt, Ryan A. Rossi, and Hoda Eldardiry. 2026. Beyond Top-k Skill Retrieval: Diversity-Aware Skill Routing for LLM Agents. *arXiv preprint arXiv:2609.05824* (2026). doi:10.48550/arXiv.2609.05824

[30] Qinzhuo Wu, Wei Liu, Jian Luan, and Bin Wang. 2024. ToolPlanner: A Tool Augmented LLM for Multi Granularity Instructions with Path Planning and Feedback. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Miami, Florida, USA, 18315–18339. doi:10.18653/v1/2024.emnlp-main.1018

[31] Duling Xu, Zheng Chen, Zaifeng Pan, Jiawei Guan, Dong Dong, Jialin Li, and Bangzheng Pu. 2026. SkillSmith: Compiling Agent Skills into Boundary-Guided Runtime Interfaces. *arXiv preprint arXiv:2605.15215* (2026). doi:10.48550/arXiv.2605.15215

[32] Fangyuan Xu, Weijia Shi, and Eunsol Choi. 2024. RECOMP: Improving Retrieval-Augmented LMs with Context Compression and Selective Augmentation. In *The Twelfth International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2024/hash/bda88ed2892f5e61c9a9bf215c566913-Abstract-Conference.html

[33] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*. https://arxiv.org/pdf/2210.03629

[34] Kun Zeng, Yu Huo, Siyu Zhang, Zi Ye, Yuecheng Zhuo, Haoyue Liu, Yuquan Lu, Junhao Wen, and Xiaoying Tang. 2026. Group of Skills: Group-Structured Skill Retrieval for Agent Skill Libraries. *arXiv preprint arXiv:2605.06978* (2026). doi:10.48550/arXiv.2605.06978

[35] YanZhao Zheng, ZhenTao Zhang, Chao Ma, YuanQiang Yu, JiHuai Zhu, Yong Wu, Tianze Xu, Baohua Dong, Hangcheng Zhu, Ruohui Huang, and Gang Yu. 2026. SkillRouter: Skill Routing for LLM Agents at Scale. *arXiv preprint arXiv:2603.22455* (2026). doi:10.48550/arXiv.2603.22455

[36] Yingli Zhou, Yaodong Su, Youran Sun, Shu Wang, Taotao Wang, Runyuan He, Yongwei Zhang, Sicong Liang, Xilin Liu, Yuchi Ma, and Yixiang Fang. 2025. In-depth Analysis of Graph-based RAG in a Unified Framework. *arXiv preprint arXiv:2503.04338* (2025). doi:10.48550/arXiv.2503.04338

A Appendix

A.1 Support-Interface Decomposition

Selecting a skill activates its accepted task support and consumes one skill slot. Coverage and community matching must therefore be evaluated on the same selected parents. Support-Interface Decomposition (SID) separates requirements with a unique supplying parent from requirements shared by several parents. It compresses the former into local coverage values and matching capacities while retaining the identities needed for joint decisions.

Model and scope. Fix the candidate pool C and the accepted support judgments. For each skill $s \in C$, let $\mathcal{R}_s$ be its supported critical requirements and $\mathcal{G}_s$ its community memberships. Selecting s activates the biclique

$$E_s = \mathcal{R}_s \times \mathcal{G}_s.$$

For $S \subseteq C$, write

$$H(S) = \bigcup_{s \in S} \mathcal{R}_s, \qquad m_H(S) = \text{match}\left(\bigcup_{s \in S} E_s\right).$$

These are the critical coverage and matching quantities in the Method. Let $H_0 = H(S_0)$ be the protected coverage, k the skill budget, and h a matching quota. We study

$$\begin{aligned} \text{OPT}(h) = \max_{S \subseteq C} \; & |H(S)| \\ \text{subject to } & |S| = k, \quad H_0 \subseteq H(S), \quad m_H(S) \geq h. \end{aligned} \tag{13}$$

An infeasible maximum is $-\infty$. The input size is N.

The model assumes unary activation: each support record has one parent. Memberships are parent-consistent, so all requirements of a parent connect to the same $\mathcal{G}_s$. Support is monotone, with no additional prerequisites, conflicts, or selected-set-dependent withdrawal. Each skill costs one slot and each critical requirement has unit value. Judgments and the candidate universe are fixed. Communities may have arbitrary overlapping memberships; the single-community construction in the prototype is a special case. Unary activation alone does not imply parent consistency.

These assumptions concern the accepted support representation. They do not assert that a semantic judgment is correct or that covered requirements will be completed during execution. The theorem also does not assume that the parameters below are small on every task.

Private support and the shared interface. Define

$$\begin{aligned} T &= \{r : |\{s \in C : r \in \mathcal{R}_s\}| \geq 2\}, \quad t = |T|, \\ A_s &= \mathcal{R}_s \cap T, \quad P_s = \mathcal{R}_s \setminus T, \quad p_s = |P_s|. \end{aligned} \tag{14}$$

The private sets P_s are pairwise disjoint. Here "private" means uniquely supplied within C, not confidential. A protected private requirement forces its unique parent into the selection. A protected requirement without a supplier, more than k forced parents, or $k > |C|$ makes the instance infeasible.

Fix a coloring χ of communities with colors $[h]$, and write $Q_s = \chi(\mathcal{G}_s)$. A local mode of parent s is a pair

$$D_s \subseteq A_s, \quad J_s \subseteq Q_s, \qquad |J_s| \leq |D_s| + p_s. \tag{15}$$

The set D_s assigns shared coverage credit to s; J_s records matching colors used by that parent. A mode contributes $|D_s| + p_s$ coverage. Selected parents must use disjoint D_s sets and disjoint J_s sets. Their ownership union must contain $H_0 \cap T$, their color union must be $[h]$, and all forced parents must be selected. Empty modes are allowed. Coverage credit need not participate in the matching.

Lemma A.1 (Fixed-selection value preservation). *Fix a parent set S and coloring χ. The maximum value of its valid local-mode assignments equals $|H(S)|$ if S protects H_0 and has an h-edge requirement–color matching. If either condition fails, no valid assignment exists.*

Proof. For any valid assignment, parent s has enough distinct requirements in $D_s \cup P_s$ to match its colors J_s. Every required edge exists by the parent biclique. Distinct parents use distinct shared requirements, private requirements, and colors. Their witnesses therefore combine into an h-edge matching. Forced parents and shared ownership protect H_0. Every credited requirement is covered, so the assigned value is at most $|H(S)|$.

Conversely, take a feasible S and an h-edge colored matching. Assign each matching edge to a selected parent that activates it. Assign ownership of a matched shared requirement to the same parent. Each requirement occurs in at most one matching edge, so these ownership decisions are compatible. Give every remaining covered shared requirement to any selected supplier. Let J_s contain the colors of the edges assigned to s. Every such edge uses an owned shared requirement or one of s's private requirements, proving Eq. (15). These modes credit every covered requirement exactly once and recover $|H(S)|$. □

Ownership is a decision witness, not the set of all physically covered requirements. For example, suppose s_1 supports r in community red, s_2 supports r in blue, and s_3 uniquely supports u in red. With all three selected, the size-two matching assigns r to blue and u to red. Crediting r irrevocably to the first processed parent would lose this witness. A valid mode assignment leaves s_1's mode empty and credits r to s_2.

Theorem A.2 (Support-Interface Decomposition). *Under the stated model, Eq.* (13) *can be solved exactly by a deterministic algorithm in*

$$2^{O(t+h)} \, \text{poly}(N)$$

time, returning an optimal skill set or reporting infeasibility. No bound on the number of private requirements, supplier frequency, skill budget, or total number of communities is required as an exponential parameter.

Proof. For a fixed coloring, process parents in order. A state $F_i(j, O, Z)$ stores the maximum private coverage from the first i parents, where j parents have been selected, $O \subseteq T$ is their assigned shared ownership, and $Z \subseteq [h]$ their used matching colors. Initialize $F_0(0, \varnothing, \varnothing) = 0$, with other states infeasible. Parent s can be skipped unless forced. Selecting it uses any mode (D, J) satisfying Eq. (15), $D \cap O = \varnothing$, and $J \cap Z = \varnothing$, and updates

$$F_{i+1}(j+1, O \cup D, Z \cup J) \leftarrow \max\{F_{i+1}(j+1, O \cup D, Z \cup J), \\ F_i(j, O, Z) + p_s\}. \tag{16}$$

At termination, require $j = k$, $H_0 \cap T \subseteq O$, and $Z = [h]$; maximize $F_{|C|}(k, O, [h]) + |O|$. Future choices depend only on these interface sets and the remaining parents. Private requirements have no other suppliers, so retaining the largest private value for a state is safe. Lemma A.1 proves exactness for the coloring, and backtracking recovers a selection.

There are at most $(k+1)2^{t+h}$ states per layer. In a state–mode pair, each shared requirement or color is already used, newly used, or unused. Hence transitions take $O(|C|k3^{t+h})$ arithmetic operations for $k \geq 1$; handle $k = 0$ directly. Value storage uses $O(k2^{t+h})$ space, excluding backtracking.

Let g be the total number of communities. An h-perfect hash family contains a coloring injective on the community endpoints of every possible h-matching. A colored matching also lifts to actual distinct communities, since different colors cannot denote the same community. Maximizing over this family thus preserves feasibility and the optimal value. The classical construction of Naor et al. [17] has size $e^h h^{O(\log h)} \log g$ and output-polynomial construction time. This contributes $2^{O(h)}\,\mathrm{poly}(N)$, giving the stated bound. Quotas exceeding the numbers of requirements or communities are rejected first. For $h = 0$, use an empty color set and omit the matching constraint; $h = 1$ needs only one coloring. □

Corollary A.3 (Coverage-first selection). *The two leading objectives of Eq.* (10)*, together with Eq.* (6) *and baseline fallback, can be solved in* $2^{O(t+v^*)}\,\mathrm{poly}(N)$ *time. Here* v^* *is the largest critical matching among admissible selections with maximum critical coverage; set* $v^* = 0$ *on fallback.*

Proof. Solve Eq. (13) for $h = 1$. If it is infeasible or its value is at most $|H_0|$, no admissible strict improvement exists. Otherwise let its value be c^*. Increase h while the optimum remains c^*, stopping at the first drop or infeasibility. The last successful quota is exactly v^*. The first failure requires at most quota $v^* + 1$, so the sum of the running times has the stated bound. The returned set may tie with others on the two leading coordinates; later coordinates of Eq. (10) are outside this guarantee. □

Correspondence to the prototype. On unary, parent-consistent inputs, accepted OCU bindings induce $\mathcal{R}_s$ and retain the parent memberships $\mathcal{G}_s$. The current selector enumerates size-k subsets of C, protects H_0, requires strict critical gain and positive critical matching, and compares the complete ordering in Eq. (10). It is an uncompressed exact implementation of the same joint decision problem. SID supplies an alternative structured solver and a complexity guarantee; the reported execution results do not measure this solver or establish a compression speedup.

A.2 Why coverage and matching require a common selection

A restricted instance already separates individual objective optimization from joint optimization. Let each substantive skill be a triple (a_s, b_s, c_s), with requirements $\{a_s, b_s\}$ from disjoint classes A, B, and one community c_s. Let P be the bipartite multigraph with one edge $a_s b_s$ per skill, and Q the requirement–community graph containing (a_s, c_s) and (b_s, c_s). Write v_P for the number of nonisolated vertices of P, and μ_P, μ_Q for the two matching numbers. Assume at least k candidate skills.

Proposition A.4 (Easy marginals, hard joint selection). *In this class,*

$$\max_{|S|=k} |H(S)| = \min(2k, k + \mu_P, v_P), \qquad \max_{|S|=k} m_H(S) = \min(k, \mu_Q).$$

Both optima are polynomial-time computable. Deciding whether one selection attains coverage $2k$ *and matching* k *is NP-complete, even when both targets are individually attainable.*

Proof. For coverage, k edges cover at most $2k$ and v_P vertices. If their nonisolated subgraph has v vertices and d components, then $k \geq v - d$ and $d \leq \mu_P$, so $v \leq k + \mu_P$. If $k \leq \mu_P$, choose k matching edges. Otherwise start with a maximum matching and add edges incident to uncovered vertices. Unmatched vertices are independent, so each addition covers one new vertex. Stop at budget k or full coverage, padding with unused edges if necessary. This attains the formula.

For matching, one parent provides only one community, giving the upper bound $\min(k, \mu_Q)$. Take that many edges of a maximum matching in Q and assign each to a supplying parent. Distinct communities imply distinct parent witnesses. Select those parents and pad to size k.

For joint hardness, replace each edge ab of a bipartite Rainbow Matching instance, colored c, by skill (a, b, c). Coverage $2k$ means the selected edges have disjoint endpoints; matching k additionally requires all k colors to differ. These conditions are precisely a rainbow matching, an established NP-complete problem [7]. A selection and a matching witness establish membership in NP.

To enforce the promise, compute the two marginal optima. If either target is unattainable, map the input to the following fixed promised no-instance with $k = 2$:

$$(a_1, b_1, c_1),\ (a_1, b_2, c_2),\ (a_2, b_1, c_2),\ (a_2, b_2, c_1).$$

Its separate optima are four covered requirements and matching two, but every four-coverage pair uses one community. Otherwise retain the original construction. This is a polynomial reduction preserving the answer. □

The same obstruction admits a nonempty protected baseline. Add a guard skill uniquely supporting q_0 in a fresh community and k empty-support dummies; use these as the size-$k+1$ baseline. Protection forces the guard. The remaining k slots retain the original

problem, with targets $2k+1$ and $k+1$. Filling dummy slots with substantive skills cannot reduce either objective. Strict improvements exist, and the coverage-first matching optimum distinguishes the joint yes- and no-instances. The hardness is inherited from Rainbow Matching, not a new hardness theorem about that classical problem.

A.3 Complexity boundaries of SID

Theorem A.5 (Independent exponential obstructions). *Assume the deterministic Exponential Time Hypothesis (ETH). No exact algorithm for Eq.* (13) *runs in time*

$$2^{o(t+h)}\,\mathrm{poly}(N).$$

With one community and $h = 1$, the dependence on t cannot be subexponential. With $t = 0$, the dependence on h cannot be subexponential. In each statement the remaining input dependence is polynomial. The same obstructions apply to the leading pair with parameter $t + \nu^*$.

Proof. By sparsification [9], it suffices to reduce sparse 3-SAT with v variables and $c = O(v)$ clauses. Form a universe of $L = v + c$ variable and clause markers. For each literal ℓ of variable i, let A_ℓ contain its variable marker and all clause markers containing ℓ. At most v literal sets cover the universe exactly when the formula is satisfiable: variable markers force one literal per variable and clause markers enforce satisfaction.

In both reductions, add a guard uniquely supporting q_0 and v empty-support dummies. Their size-$v+1$ baseline protects $H_0 = \{q_0\}$, forcing the guard in every feasible set.

For the shared-requirement obstruction, give literal parent ℓ support A_ℓ, and give all parents the same community. Set $h = 1$. The maximum coverage is $L + 1$ iff the formula is satisfiable, while every feasible selection has matching one. Variable markers each have two suppliers, so $v \le t \le L$. Each literal covers a variable marker, so a strict gain over the baseline exists. A subexponential algorithm in t, or in $t + \nu^*$ with $\nu^* = 1$, would decide sparse 3-SAT in subexponential time.

For the community obstruction, give every literal parent L unique private requirements and memberships A_ℓ. The guard has a fresh community. Thus $t = 0$. With j literal parents, coverage is $1 + jL$, so primary optima select v literals. For every such selection,

$$m_H(S) = 1 + \left| \bigcup_{\ell \in S \setminus \{\mathrm{guard}\}} A_\ell \right|.$$

The right side bounds the number of available communities. To attain it, assign each active community to a selected literal parent containing it; each parent receives at most L communities and has L private requirements to match them. Add the guard's edge. Quota $h = L + 1$ is feasible iff the formula is satisfiable. Equivalently, the leading matching value among maximum-coverage sets is $L + 1$ iff satisfiable, and is always $O(v)$. Both claimed subexponential bounds would therefore contradict ETH. The constructions have polynomial explicit size. □

Under ETH, the dependence on shared requirements and matching quota cannot both be made subexponential in the general support model. Separate constructions establish an obstruction in t with $h = 1$, and in h with $t = 0$; the latter uses multiple community memberships.

A.4 Implementation settings

Retrieval uses the two highest-scoring communities and 30 source blocks, with fusion weights $\alpha = 0.6$, $\beta = 0.4$, $\gamma = 0.15$, and $\lambda = 0.10$. A constant score vector is normalized to 0.5. Requirement–OCU nomination retains at most 10 candidates per requirement and two per skill, using unigram and bigram TF–IDF with English stop words removed.

A.5 Results reporting

On SkillsBench with Grok, the available official task rewards total 51.2300. They cover 86 tasks evaluated in two batches; one additional task has no official score because the scoring service rejected its credential.

The L_1-only control retains the full second stage. Table 2 reports 33.7130 from the second executed cohort. This includes 86 official rewards totaling 33.3830 and a fuzz-task score of 0.3300 computed from the unchanged six-group scoring rule; the final verifier did not complete for that task.

For Grok AgentSkillOS, the five fixed-selection methods use the 200-skill pool, Grok-4.6, a prepared public-tool runtime, and a 1,800-second task budget. Each row contains 30 official scores. The comparison evaluates each complete system with its own retrieval, context construction, and execution guidance.

Native scores 2903.05 with Codex and 2651.22 with Grok on AgentSkillOS. The Codex result includes nine tasks exceeding 1,800 seconds; six Grok task results use a revised planning and execution configuration. Native serves as the benchmark-workflow reference.

A.6 Support and input-change definitions

Across 87 Codex/GPT-5.5 SkillsBench tasks, 456 of 4,820 nominated requirement–OCU pairs receive positive judgments, and 314 are admitted after validation. These are pair counts, not task counts. Guidance denotes nonempty guide text; a set change compares skill membership with the initial selection, ignoring order. The 28 guided tasks comprise 11 changed-set and 17 unchanged-set tasks.

A.7 Reproducibility resources

The revision bundle contains the result inventory, table-generation scripts, figure sources, and mappings from reported results to source records.

Figure credits. Icons: feen, Chanut-is-Industries, rukanicon, Magnific, and Freepik (Flaticon attribution licenses). The Python mark is licensed for editorial academic use. Asset manifests list source pages and hashes; diagram text and geometry are editable vectors.